

elements of type 'dtype' from 'string' and generates an array. 'String' will be interpreted as a binary if 'sep', a string, is not provided, and as ASCII otherwise. I would like to add a bit more about *fromstring()*. This function needs the input string size to be a multiple of the element size. Unless specified, the array created using this function will be of dtype 'float64', which requires 8 bytes for representation. Consider the example given below:

```
>>>a=n.fromstring('123')
```

The above statement intends to generate an array from the string '123'. But it will generate an error message since its length is not a multiple of 8.

```
>>>a=n.fromstring('12345678')
```

The above statement will successfully generate an array as given below:

```
>>>a
array([ 6.82132005e-38])
```

Consider the example given below:

```
>>>a=n.fromstring('123456',dtype='S2',count=2)
```

Here, dtype is specified as 'S2'. So the array 'a' will contain elements of length 2.

```
>>>a
array(['12', '34'],
      dtype='<S2')
```

We can see that 'a' contains only two elements since the count given in *fromstring()* is 2.

loadtxt([name[,dtype]],comments[,skiprows][,delimiter][,converters[,usecol],...]): This returns an array containing elements formed from the data in the file. The contents of an input file, say *loadtxt.txt*, are given below:

```
#this is comment line
abc def ghi
jkl mno pqr
```

Use the function shown below:

```
>>>n.loadtxt('/home/abc/Desktop/loadtxt.txt',dtype='S3')
array([[ 'abc', 'def', 'ghi'],
       [ 'jkl', 'mno', 'pqr']],
      dtype='<S3')
```

We can see in the output that the comment statement has automatically been eliminated. Before applying this function,

we must make sure that all rows in the file contain an equal number of strings. We can specify in the comments option of *loadtxt()* which character will mark the beginning of the comments. By default, it is the '#' symbol. The *skiprows* option will help to skip the first 'skiprows' lines in the input file.

arange(start, stop, step[,dtype]): This returns an array containing elements within a range.

There are many more functions that help in generating arrays. They are detailed in the official site *scipy.org*.

Functions associated with arrays

We have written programs to find trace, to sort elements, to find the index of non-zero elements, to multiply two matrices, etc. We know how lengthy these programs are, if written in C. Each of these tasks can be finished with a single statement using NumPy. The description of a few functions that are used is given below. A majority of the functions associated with an array return an array.

nonzero(a): This returns a tuple containing the indices of non-zero elements in the array.

```
>>>a=n.array([[0,0,2],[3,0,0],[0,0,0]])
>>>n.nonzero(a)
(array([0, 1]), array([2, 0]))
```

We can see in the definition of 'a' that indices of non-zero elements in it are [0,2] and [1,0]. Each element in the result of *nonzero()* is an array containing the index position of the non-zero element in that dimension. In this case, the first array contains row numbers and the second array contains column numbers of non-zero elements. If we had a third dimension in the input array, the tuple would have contained one more element showing the positions of non-zero elements in that dimension.

```
>>>a[n.nonzero(a)]
array([2, 3])
```

The above code shows us how to retrieve the non-zero elements from the array.

transpose(a[, axes]): This returns a new ndarray after performing a permutation on dimensions. The code segment given below shows a 3D array and its transpose.

```
>>>a=n.array((((1,2,3),(4,5,6)),((7,8,9),(0,1,2))))
>>>a
array([[[[1, 2, 3],
         [4, 5, 6]],
        [[7, 8, 9],
         [0, 1, 2]]],
       [[1, 7],
        [4, 0]]],
      dtype=object)
```